

Task 'Designing Real World Application with SOLID Principles' Unit 2

Designing Real World Application with SOLID Principles

Using the application of the SOLID principles, it is possible to improve flexibility and maintainability of software systems. The original shopping system combined product storage, price calculation, and payment handling into one class, making the design difficult to extend. Refactoring the code into smaller classes creates a cleaner and more scalable solution.

(Marcotte and Zebdi, 2022).

```
```python
```

```
from abc import ABC, abstractmethod
```

```
class Product:
```

```
 def __init__(self, name, price):
```

```
 self.name = name
```

```
 self.price = price
```

```
class Cart:
```

```
 def __init__(self):
```

```
 self.items = []
```

## Task 'Designing Real World Application with SOLID Principles' Unit 2

```
def add_item(self, product):
```

```
 self.items.append(product)
```

```
def total(self):
```

```
 return sum(i.price for i in self.items)
```

```
class Discount(ABC):
```

```
 @abstractmethod
```

```
 def apply(self, total):
```

```
 pass
```

```
class TenPercentDiscount(Discount):
```

```
 def apply(self, total):
```

```
 return total * 0.90
```

```
class PaymentMethod(ABC):
```

```
 @abstractmethod
```

```
 def pay(self, amount):
```

```
 pass
```

## Task 'Designing Real World Application with SOLID Principles' Unit 2

```
class DebitCard(PaymentMethod):

 def pay(self, amount):

 print(f"Paid €{amount} by Debit Card")

class Checkout:

 def __init__(self, payment_method, discount):

 self.payment_method = payment_method

 self.discount = discount

 def process(self, cart):

 total = cart.total()

 final_total = self.discount.apply(total)

 self.payment_method.pay(final_total)

cart = Cart()

cart.add_item(Product("Laptop Bag", 80))

cart.add_item(Product("Wireless Mouse", 40))
```

## Task 'Designing Real World Application with SOLID Principles' Unit 2

```
Checkout(DebitCard(), TenPercentDiscount()).process(cart)
```

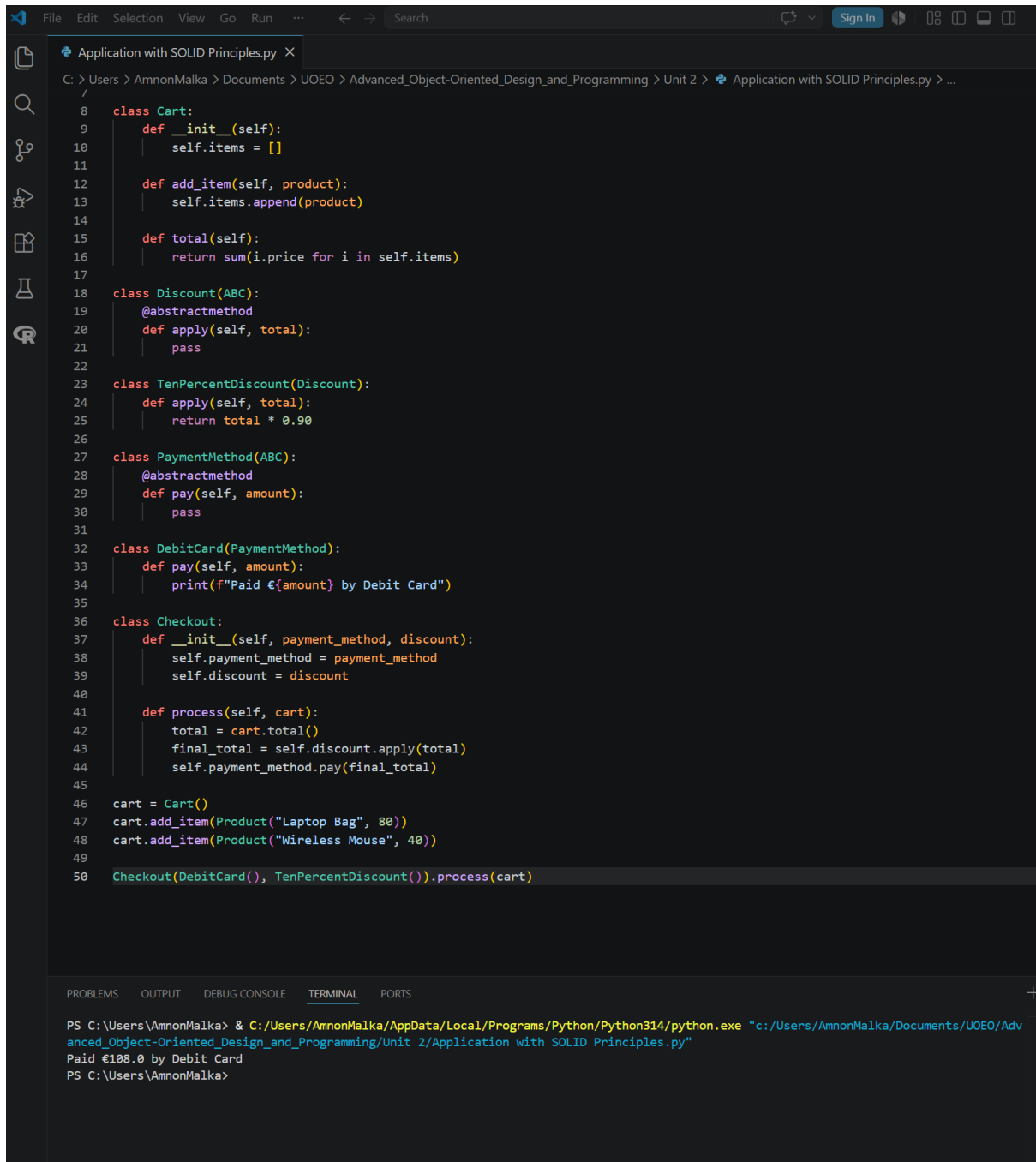
...

### Results

The refactored code is developed to reflect the requirements of the Single Responsibility Principle, where each class has one clear responsibility, the Cart manages products, Discount manages price reductions, and `PaymentMethod` processes the payments, the Open/Closed Principle is satisfied because new discount or payment types can now be added through new subclasses, without modifying existing code. The Liskov Substitution Principle is preserved by the fact that any subclass of `PaymentMethod` can substitute the base type, while preserving program correctness. In addition, Interface Segregation Principle is achieved through small, focused interfaces that avoid forcing classes to implement unnecessary methods. The Dependency Inversion Principle is enforced, resulting in the Checkout class relies on abstractions rather than concrete implementations, thus facilitating loose coupling and improving flexibility. This design showcases a proper object-oriented design by providing consistency between naming and behavior, improving clarity, and sustaining future extensibility without modifying existing logic.

(Marcotte and Zebdi, 2022).

## Task 'Designing Real World Application with SOLID Principles' Unit 2



```
8 class Cart:
9 def __init__(self):
10 self.items = []
11
12 def add_item(self, product):
13 self.items.append(product)
14
15 def total(self):
16 return sum(i.price for i in self.items)
17
18 class Discount(ABC):
19 @abstractmethod
20 def apply(self, total):
21 pass
22
23 class TenPercentDiscount(Discount):
24 def apply(self, total):
25 return total * 0.90
26
27 class PaymentMethod(ABC):
28 @abstractmethod
29 def pay(self, amount):
30 pass
31
32 class DebitCard(PaymentMethod):
33 def pay(self, amount):
34 print(f"Paid €{amount} by Debit Card")
35
36 class Checkout:
37 def __init__(self, payment_method, discount):
38 self.payment_method = payment_method
39 self.discount = discount
40
41 def process(self, cart):
42 total = cart.total()
43 final_total = self.discount.apply(total)
44 self.payment_method.pay(final_total)
45
46 cart = Cart()
47 cart.add_item(Product("Laptop Bag", 80))
48 cart.add_item(Product("Wireless Mouse", 40))
49
50 Checkout(DebitCard(), TenPercentDiscount()).process(cart)
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
PS C:\Users\AmnonMalka> & C:/Users/AmnonMalka/AppData/Local/Programs/Python/Python314/python.exe "c:/Users/AmnonMalka/Documents/UOEO/Advanced_Object-Oriented_Design_and_Programming/Unit 2/Application with SOLID Principles.py"
Paid €108.0 by Debit Card
PS C:\Users\AmnonMalka>
```

Figure 1: Refactoring Results

## References

## Task 'Designing Real World Application with SOLID Principles' Unit 2

- Marcotte, C.-H. & Zebdi, A. (2022) *An Atypical ASP. NET Core 6 Design Patterns Guide: A SOLID Adventure into Architectural Principles and Design Patterns Using . NET 6 and C# 10*. 2nd edition. Birmingham: Packt Publishing, Limited.

### Use of Digital Tools

This document has been produced exclusively for educational purposes. Referenced content, copyrights, and trademarks, remain the sole property of the respective owner. ChatGPT 5.5 was used for some proofreading and clarity as well as literature exploration, language refinement, coding and programming techniques research. All academic writing, analysis, argumentation, and conclusions represent the original work of the author. AI tools were used for exploration, same way as academic literature exploration. No AI-generated content or code was directly copied into the final submission. Any use of content mentioned in this document is properly referenced to its original author and was used responsibly to ensure integrity and originality. All academic writing, analysis, argumentation, and conclusions represent my original work. I have used Artificial Intelligence responsibly and ethically, in accordance with the assignment brief. All AI use has been declared and evidenced.